SIMATS ENGINEERING

CO2_AT3 Error Analysis Task

COURSE NAME: Software Testing **COURSE CODE: CSA3712**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 20%

Sl. No	Questions	Bloom's Level
1	A tester designs test cases for an online shopping system using equivalence partitioning but incorrectly groups input ranges (e.g., valid and invalid inputs mixed in one class). Additionally, boundary values are not considered. Analyze the given test design and identify all errors in equivalence class partitioning and boundary value analysis. Examine the root causes of these mistakes with proper reasoning. Propose corrected test cases with proper classification and boundary coverage. Justify how your corrections improve defect detection effectiveness.	3
2	A QA engineer performs random testing on a banking application and claims high coverage without using any structured techniques. Critical defects in transaction limits and boundary conditions are missed. Evaluate the testing approach and identify errors in strategy and coverage assumptions. Analyze the root causes behind missing critical defects. Suggest appropriate black box testing techniques to correct the approach. Justify how these techniques ensure better test adequacy and reliability.	3
3	A developer performs white box testing but creates an incomplete control flow graph, missing decision nodes and loops, resulting in poor path coverage. Analyze the errors in the control flow graph and path testing approach. Identify the root causes for inadequate test coverage. Propose corrections by constructing a proper control flow representation and identifying all independent paths. Justify how improved path testing ensures better coverage and defect detection.	5

4	A testing team reports 100% statement coverage for a module but fails to detect logical errors in conditional statements. Evaluate the adequacy of the testing performed and identify gaps in coverage criteria. Analyze why statement coverage alone was insufficient. Recommend improved white box testing strategies such as branch or path coverage. Justify how the revised approach enhances overall test effectiveness and error detection.	5
5	A tester uses Equivalence Class Partitioning for an age-validation field (0-100) but defines only two classes: Negative numbers and Positive numbers. Analyze the errors in this partitioning strategy. Identify why this fails to account for valid vs. invalid age limits and the absence of Boundary Value Analysis. Propose a corrected set of partitions and justify how they ensure more robust input validation.	3
6	A developer claims full Condition Coverage for a complex `if(A && B	
7	A QA team applies Decision Table Testing for a discount system but misses several combinations of "Premium Member" and "Coupon Applied," leading to price calculation errors. Analyze the errors in the construction of the decision table. Identify why the current approach is incomplete. Propose a revised, exhaustive decision table and justify its effectiveness in preventing production defects.	3
8	During unit testing of a looping structure, a tester checks only the first and last iterations (n=1 and n=max). Defects occurring at n=0 and n=max+1 are missed. Evaluate the errors in this Loop Testing strategy. Analyze the root causes behind these omissions. Recommend a corrected approach for testing simple and nested loops, and justify how it ensures path integrity.	5
9	An engineer utilizes Error Guessing as the primary testing technique for a safety-critical braking system, ignoring structured black box methods. Identify the errors in relying solely on ad-hoc techniques for high-risk modules. Analyze the impact on test adequacy. Suggest a combination of BVA and State Transition Testing to fix the strategy, justifying the increased safety assurance.	5
10	A tester designs a Control Flow Graph (CFG) for a module but fails to represent the "else" branches of multiple nested if statements, leading to 100% statement coverage but 0% branch coverage on error paths. Analyze the errors in graph construction. Identify the root causes for the missing nodes. Propose a corrected CFG and independent paths to ensure branch adequacy.	5

Code of Conduct:

I Peta Harsha Vardhan(192311039) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Question 1

A tester designs test cases for an online shopping system using equivalence partitioning but incorrectly groups input ranges (e.g., valid and invalid inputs mixed in one class). Additionally, boundary values are not considered. Analyze the given test design and identify all errors in equivalence class partitioning and boundary value analysis. Examine the root causes of these mistakes with proper reasoning. Propose corrected test cases with proper classification and boundary coverage. Justify how your corrections improve defect detection effectiveness.

Answer

Assumption: the paper does not state the field, so the "Quantity" field of the shopping cart is taken, with the rule "a customer may order 1 to 99 items of a product". The faulty design groups the inputs into two classes: "0 to 50" and "51 to 100", with one random value from each (for example 25 and 75).

1. Errors Identified

No.	Error in the given test design	Effect
1	Valid and invalid values are mixed in one class: "0 to 50" contains 0 (invalid) with 1 to 50 (valid), and "51 to 100" contains 100 (invalid) with 51 to 99 (valid)	Members of a class do not behave alike, so one value cannot represent the class (breaks the basic rule of equivalence partitioning)
2	No separate invalid classes: negative numbers, decimals, non-numeric text, blank and special characters	Input validation is never tested
3	Classes are not derived from the requirement (1 to 99); the range 0 to 100 was assumed	The test set does not match the specification
4	No boundary values: 0, 1, 2, 98, 99 and 100 are missing	Off-by-one defects such as "quantity > 1" or "quantity < 99" remain undetected
5	Only one representative per class and no expected results	Tests are not verifiable or traceable

2. Root Cause Analysis

- The tester partitioned the numbers by convenient round values (50 and 100), not by the behaviour defined in the requirement.
- Lack of understanding of the rule that each class must be homogeneous: all members are either valid or invalid, and are processed in the same way.
- Equivalence partitioning and boundary value analysis were treated as separate and optional; BVA, which complements EP, was never applied.
- Only the "valid" side of the specification was considered (positive thinking), so invalid classes were not identified.
- No review of the test design, no traceability to requirements and no checklist of invalid data types.

3. Corrected Test Design

Step 1: Equivalence classes (each class is homogeneous)

Class	Description	Representative value	Expected result
V1 (valid)	1 to 99	50	Accepted
I1 (invalid)	Less than 1 (zero and negatives)	-5	Error message

Class	Description	Representative value	Expected result
I2 (invalid)	Greater than 99	150	Error message: maximum is 99
I3 (invalid)	Decimal number	2.5	Error message: whole number required
I4 (invalid)	Non-numeric / special characters	abc, #@	Error message
I5 (invalid)	Blank (empty)	(no entry)	Error message: mandatory field

Step 2: Boundary values (BVA: just below, on and just above each boundary)

Test ID	Input	Boundary	Expected result
TC1	0	Just below minimum	Rejected
TC2	1	Minimum	Accepted
TC3	2	Just above minimum	Accepted
TC4	98	Just below maximum	Accepted
TC5	99	Maximum	Accepted
TC6	100	Just above maximum	Rejected

4. Justification

- Homogeneous classes mean that one test per class is enough to represent the whole class, so coverage is complete with few cases (11 to 12 tests).
- Separate invalid classes test the validation logic, which is the most common source of failures in an online shop (negative quantities cause negative bills).
- Boundary values reveal the defects that cluster at the edges, such as "<" instead of "<=", that the representative values 25 and 75 can never reveal.
- Classes are derived from the requirement, so every rule is traceable, and the set is repeatable and reviewable.
- Defect detection improves because the revised set finds all the defect types missed by the original design: wrong boundaries, missing validation and wrong data types.

Question 2

A QA engineer performs random testing on a banking application and claims high coverage without using any structured techniques. Critical defects in transaction limits and boundary conditions are missed. Evaluate the testing approach and identify errors in strategy and coverage assumptions. Analyze the root causes behind missing critical defects. Suggest appropriate black box testing techniques to correct the approach. Justify how these techniques ensure better test adequacy and reliability.

Answer

Assumed requirement for illustration: a fund transfer is allowed from Rs.100 to Rs.50,000 per transaction, the daily limit is Rs.1,00,000, and the balance must cover the amount.

1. Errors in Strategy and Coverage Assumptions

- **Coverage was claimed without any measure:** the number of tests executed or the time spent is not coverage. Coverage must be measured against requirements, input classes or code.
- Random values fall mostly in the "middle" of the input domain (Rs.2,000, Rs.5,000), so the probability of hitting a boundary such as Rs.50,000 or Rs.50,001 is very small.
- No test design, no expected results (no oracle) and no record of the inputs used, so tests are not repeatable and failures cannot be reproduced.
- Only valid, typical data were used; negative, zero, over-limit and invalid account cases were ignored.
- Combinations of rules (balance, per-transaction limit, daily limit, account status) were never tested systematically.
- No traceability between requirements and tests, so untested requirements are invisible.

2. Root Cause Analysis

- The belief that more random tests means more coverage, which is wrong because exhaustive testing is impossible and effort must be directed by technique and risk.
- Lack of knowledge or use of black-box techniques (equivalence partitioning, boundary value analysis, decision tables, state transition).
- Defects cluster at boundaries and at rule combinations; random selection is not designed to reach them.
- **No risk analysis:** transaction limits are high-risk, yet received no extra attention.
- Absence of test adequacy criteria and of test review.

3. Corrected Approach with Black-Box Techniques

Technique	Application to the banking application	Example test inputs
Equivalence partitioning	Classes: below minimum, valid range, above maximum, invalid data type	Rs.50 (invalid), Rs.5,000 (valid), Rs.60,000 (invalid), "abc"
Boundary value analysis	Per-transaction limit and minimum amount	99, 100, 101, 49,999, 50,000, 50,001
Decision table	Combinations: balance sufficient, within per-transaction limit, within daily limit, account active	8 or more rules, for example sufficient balance but daily limit exceeded: rejected
State transition	Account states: Active, Dormant, Blocked, Closed	Transfer from a blocked account must be rejected; unblock then transfer succeeds
Use case / negative testing	Complete transfer flow, timeout, duplicate submission	Submit twice: only one debit

4. Justification

- **Test adequacy:** Coverage is now measurable (all classes, all boundaries, all decision rules and all state transitions covered), so "high coverage" can be demonstrated, not just claimed.
- **Reliability:** Boundary and rule-combination defects, which are the critical ones in transaction limits, are targeted directly.
- **Efficiency:** A small set of about 30 designed cases outperforms hundreds of random cases.

- **Repeatability and traceability:** Each test has an ID, input, expected result and a link to its requirement, which supports regression testing and audits.

Question 3

A developer performs white box testing but creates an incomplete control flow graph, missing decision nodes and loops, resulting in poor path coverage. Analyze the errors in the control flow graph and path testing approach. Identify the root causes for inadequate test coverage. Propose corrections by constructing a proper control flow representation and identifying all independent paths. Justify how improved path testing ensures better coverage and defect detection.

Answer

Sample module under test (positive-sum program):

```
1  read(n); sum = 0; neg = 0; i = 1;
2  while (i <= n)
3  {   if (a[i] > 0)
4         sum = sum + a[i];
5     else
6         neg = neg + 1;
7     i = i + 1;
8  }
9  print(sum, neg);
```

1. Errors in the Given Control Flow Graph and Path Testing

- The loop condition (i <= n) was not drawn as a decision node, and the back edge from the end of the loop body to the loop condition is missing, so the CFG looks like straight-line code.
- The if-else decision (a[i] > 0) was not represented with its two outgoing edges, so one of the branches (the else part) was omitted.
- The join node where the if and else branches merge (i = i + 1) is missing.
- The loop-skip edge (n = 0, loop never entered) is missing, so the path that bypasses the loop is not considered.
- Because of this, only one or two paths were identified and the cyclomatic complexity was wrongly calculated.

2. Root Causes

- Confusion of the CFG with a flowchart or a simple statement list, without applying the rules: one node for each sequence of statements, one decision node for each condition, edges for each outcome.
- Loops were not recognised as decisions with a feedback edge.
- No systematic use of the formula V(G) = E - N + 2 or the decision count + 1 to cross-check the graph.
- No review of the CFG against the code, and no tool support.

3. Corrected Control Flow Graph

Node	Statements	Type
N1	1	Process (initialisation)
N2	2 while (i <= n)	Decision (loop)

Node	Statements	Type
N3	3 if (a[i] > 0)	Decision
N4	4 sum = sum + a[i]	Process (then)
N5	6 neg = neg + 1	Process (else)
N6	7 i = i + 1	Process (join, then back to N2)
N7	9 print	Exit

Edges: N1-N2, N2-N3 (true), N2-N7 (false), N3-N4 (true), N3-N5 (false), N4-N6, N5-N6, N6-N2 (back edge). So E = 8, N = 7 and decision nodes P = 2.

Cyclomatic complexity V(G) = E - N + 2 = 8 - 7 + 2 = 3, and also P + 1 = 3. Therefore there are 3 independent paths.

Path	Sequence of nodes	Test data	Expected result
P1	N1-N2-N7 (loop skipped)	n = 0	sum = 0, neg = 0
P2	N1-N2-N3-N4-N6-N2-N7 (positive element)	n = 1, a[1] = 5	sum = 5, neg = 0
P3	N1-N2-N3-N5-N6-N2-N7 (non-positive element)	n = 1, a[1] = -3	sum = 0, neg = 1

4. Justification

- The complete CFG shows every decision and loop, so no executable path is hidden.
- A basis set of independent paths, each adding one new edge, guarantees that all statements and all branches (both outcomes of both decisions) are executed, at least once.
- The number of independent paths (V(G)) gives a measurable minimum number of tests, so coverage can be demonstrated.
- Defects in the loop condition (< instead of <=), in the loop-skip case and in each branch are exposed, and high-complexity modules can be identified for extra testing or refactoring.

Question 4

A testing team reports 100% statement coverage for a module but fails to detect logical errors in conditional statements. Evaluate the adequacy of the testing performed and identify gaps in coverage criteria. Analyze why statement coverage alone was insufficient. Recommend improved white box testing strategies such as branch or path coverage. Justify how the revised approach enhances overall test effectiveness and error detection.

Answer

Module used for illustration (loan eligibility):

```
1  eligible = false;
2  if (age >= 18 && income >= 25000)
3      eligible = true;
4  return eligible;
```

1. Evaluation of the Testing Performed and Gaps

A single test, for example age = 30 and income = 40000, executes statements 1, 2, 3 and 4, so statement coverage is 100%. But:

- The decision at line 2 was evaluated only to TRUE; the FALSE outcome (branch coverage 1 of 2 = 50%) was never exercised.
- The two conditions (age and income) were never FALSE, so condition coverage is 0% for the false outcomes.
- Boundary values (age = 17, 18; income = 24,999, 25,000) were never tested.
- **Logical defects remain hidden:** ">=" written instead of ">", "&&" written instead of "||", or a wrong constant.

2. Why Statement Coverage Alone Was Insufficient

- It only checks that every statement is executed once; it does not check the outcomes of decisions.
- An "if" without an else has an implicit empty false branch containing no statement, so it can be skipped completely and the coverage figure stays 100%.
- It does not check the individual conditions in a compound expression, nor the order of evaluation.
- **It is the weakest white-box criterion:** it does not subsume branch, condition or path coverage.

3. Improved White Box Strategies

Criterion	What is required	Test cases (age, income)	Result
Branch (decision) coverage	TRUE and FALSE outcome of the if	(30, 40000), (16, 40000)	TRUE / FALSE
Condition coverage	Each condition TRUE and FALSE at least once	(30, 40000), (16, 10000)	TRUE / FALSE
MC/DC	Each condition independently changes the outcome	(30, 40000), (16, 40000), (30, 10000)	TRUE, FALSE, FALSE
Boundary additions	Edges of both conditions	(18, 25000), (17, 25000), (18, 24999)	TRUE, FALSE, FALSE
Path coverage	All distinct paths (2 paths here)	Covered by branch tests	Both paths

4. Justification

- Branch coverage subsumes statement coverage, and it forces the false outcome of every decision to be tested, which exposes missing-else and wrong-condition defects.
- Condition coverage and MC/DC check each atomic condition, so errors such as "&&" instead of "||" are detected.
- Boundary tests expose relational operator errors (> against >=).
- The result is a stronger, measurable criterion of adequacy, so the reported coverage reflects real logical verification and not only execution of statements.

Question 5

A tester uses Equivalence Class Partitioning for an age-validation field (0-100) but defines only two classes: Negative numbers and Positive numbers. Analyze the errors in this partitioning strategy. Identify why this

fails to account for valid vs. invalid age limits and the absence of Boundary Value Analysis. Propose a corrected set of partitions and justify how they ensure more robust input validation.

Answer

1. Errors in the Partitioning Strategy

No.	Error	Explanation
1	Classes are based on sign, not on the specification (valid range 0 to 100)	"Positive numbers" mixes valid ages (1 to 100) with invalid ones (101 and above, for example 150)
2	Zero is in neither class	0 is a valid age (0 to 100), but it is neither negative nor positive, so the boundary is unclassified
3	No distinction between valid and invalid classes	The test cannot show whether the limits 0 and 100 are enforced
4	Other invalid inputs are ignored: decimals (25.5), text ("abc"), special characters, blank	Validation of data type and mandatory field remains untested
5	No boundary value analysis	Defects at -1, 0, 1, 99, 100 and 101 remain hidden

2. Why the Strategy Fails

Equivalence classes must be homogeneous: every value in a class must be treated in the same way by the program. In the given design, the class "Positive numbers" contains both accepted and rejected values, so a single representative (for example 30) says nothing about 150. The limits of the valid range are never examined, so a defect such as "age < 100" instead of "age <= 100" is not found. The tester split the domain by mathematical sign and not by the validation rule, and did not apply boundary value analysis, which complements equivalence partitioning.

3. Corrected Partitions

Class	Range / description	Type	Representative	Expected result
EC1	0 to 100	Valid	45	Accepted
EC2	Less than 0	Invalid	-7	Rejected
EC3	Greater than 100	Invalid	150	Rejected
EC4	Decimal number	Invalid	25.5	Rejected
EC5	Non-numeric / special characters	Invalid	abc, @#	Rejected
EC6	Blank / null	Invalid	(empty)	Rejected: mandatory

Boundary values: -1 (rejected), 0 (accepted), 1 (accepted), 99 (accepted), 100 (accepted), 101 (rejected).

4. Justification

- Each class is homogeneous, so one value represents it, and all valid and invalid situations of the specification are covered.
- The separate invalid classes ensure that data type, range and mandatory checks all work, which makes the validation robust against wrong or malicious data.
- Boundary values detect off-by-one errors at 0 and 100, where defects are most likely.

- Total of 12 tests (6 classes plus 6 boundary values, with overlaps reducible) gives complete coverage with minimum effort.

Question 6

A developer claims full Condition Coverage for a complex `if(A && B (the question text is incomplete in the question paper).

Answer

Assumption: the complete condition is taken as the common form if ((A && B) || C), where A, B and C are Boolean conditions. The reasoning applies equally to any compound condition.

1. Errors in the Claim of Full Condition Coverage

Suppose the developer used two tests: T1 (A, B, C) = (True, True, False) and T2 = (False, False, True). Each of A, B and C has been TRUE once and FALSE once, so the developer claims full condition coverage. But:

Test	A	B	C	Decision value
T1	True	True	False	True
T2	False	False	True	True

- **The decision evaluates to TRUE in both tests:** the FALSE outcome of the if statement is never executed, so decision (branch) coverage is only 50%. Condition coverage does not guarantee decision coverage.
- With short-circuit evaluation, when A is False, B is not evaluated at all in T2, so B = False was never really exercised; the claim is false for the program as actually executed.
- Condition coverage does not show that each condition independently affects the result, so an error such as "&&" instead of "||" can remain hidden.
- The combinations of conditions are not covered: only 2 of 8 combinations were run.

2. Root Causes

- Confusion between condition coverage (each atomic condition TRUE and FALSE), decision coverage (whole expression TRUE and FALSE) and condition/decision coverage.
- Ignoring short-circuit evaluation (&& and || skip the remaining operands).
- Test values were chosen to flip each variable once, without computing the resulting decision value or the effect of each condition.
- No truth table analysis and no tool to measure coverage.

3. Corrected Test Set (MC/DC)

Modified Condition/Decision Coverage requires that each condition is shown to independently change the decision, with n + 1 = 4 tests for 3 conditions:

Test	A	B	C	(A && B) || C	Shows independence of
T1	True	True	False	True	A and B (paired with T2, T3)
T2	False	True	False	False	A (T1 against T2)
T3	True	False	False	False	B (T1 against T3)
T4	True	False	True	True	C (T3 against T4)

This set also gives decision coverage (TRUE and FALSE outcomes) and condition coverage (each of A, B and C True and False).

4. Justification

- MC/DC is stronger than condition or decision coverage alone, and it needs only n + 1 tests, not all 2^n combinations (8 here), so it is practical for complex conditions.
- Each condition is proven to matter, so operator errors and missing conditions are detected.
- It subsumes decision and condition coverage and is required by safety standards (for example DO-178C for airborne software), so it gives a reliable and defensible adequacy claim.

Question 7

A QA team applies Decision Table Testing for a discount system but misses several combinations of "Premium Member" and "Coupon Applied," leading to price calculation errors. Analyze the errors in the construction of the decision table. Identify why the current approach is incomplete. Propose a revised, exhaustive decision table and justify its effectiveness in preventing production defects.

Answer

Assumed discount rules: a Premium Member receives 10% off; a coupon gives an extra 5% only when the order value is Rs.1,000 or more; the two discounts combine (maximum 15%); otherwise no discount.

1. Errors in the Construction of the Decision Table

- The table lists only some combinations (for example Premium + Coupon, Premium only and None), and the combination "Non-premium member with Coupon" is missing, together with the cases of the order-value condition.
- **The number of rules is not 2^n:** with three conditions there must be 8 rules, but fewer were written, so the table is incomplete.
- The condition "order value >= Rs.1,000" that decides whether a coupon is accepted was not included as a condition.
- Actions for rejected coupons (error message, no discount) were not specified, so the expected results are unclear.
- Redundant rules were not merged with "don't care" entries, and the table was not checked for conflicting or missing rules.

2. Why the Current Approach Is Incomplete

A decision table is complete only if every combination of condition values has a rule. Missing the combination (Non-premium, Coupon applied) means that the 5% coupon discount for ordinary customers was never tested, and the interaction of both discounts was not tested at the order-value boundary. This is exactly where price calculation errors occur, because the program has to evaluate several rules together.

3. Revised Exhaustive Decision Table

Conditions / Actions	**R1**	**R2**	**R3**	**R4**	**R5**	**R6**	**R7**	**R8**
C1: Premium member	Y	Y	Y	Y	N	N	N	N
C2: Coupon applied	Y	Y	N	N	Y	Y	N	N
C3: Order value >= Rs.1,000	Y	N	Y	N	Y	N	Y	N

Conditions / Actions	R1	R2	R3	R4	R5	R6	R7	R8
A1: Discount given	15%	10%	10%	10%	5%	0%	0%	0%
A2: Message "coupon not valid"	No	Yes	No	No	No	Yes	No	No

Optimised form: R3 and R4 merge into (Y, N, -) = 10%, and R7 and R8 merge into (N, N, -) = 0%, giving 6 rules. Each rule becomes one test case, for example R5: non-premium member, coupon, order Rs.1,500 gives Rs.75 discount; boundary tests with Rs.999 and Rs.1,000 are added for C3.

4. Justification

- All 2^3 = 8 combinations are covered, so no business rule combination is left untested.
- The table exposes missing or contradictory requirements (for example, the rule for a coupon below the minimum value) before coding.
- Test cases derive directly from the rules, so they are complete, traceable and easy to review.
- Price calculation errors in rule interactions are found before release, which prevents wrong billing and customer complaints in production.

Question 8

During unit testing of a looping structure, a tester checks only the first and last iterations (n=1 and n=max). Defects occurring at n=0 and n=max+1 are missed. Evaluate the errors in this Loop Testing strategy. Analyze the root causes behind these omissions. Recommend a corrected approach for testing simple and nested loops, and justify how it ensures path integrity.

Answer

Example: a loop that processes up to max = 100 records: for (i = 1; i <= n; i++) { process(record[i]); }

1. Errors in the Loop Testing Strategy

- The zero-iteration case (n = 0), where the loop body is skipped, was not tested; defects such as a null result, uninitialised variable or exception appear here.
- The case n = max + 1 (one more than the maximum) was not tested, so the program's protection against overflow (array index out of range, buffer overflow) is unverified.
- Typical values (two iterations, an intermediate number, max - 1) were not tested, so defects that appear only from the second pass (state carried from one iteration to the next) are missed.
- Only the boundaries of the valid values were used, but not the invalid side of the boundary; and no loop is checked for termination.

2. Root Causes

- The tester equated "loop testing" with checking the first and last iterations, which is only a part of the loop test technique.
- Boundary value thinking was applied only to valid values and not to the values just outside the range (0 and max + 1).
- The control flow graph was not drawn, so the different paths (skip the loop, enter once, repeat, leave) were not identified.
- Programmers' common mistakes at loop limits (off-by-one, "<" for "<=", wrong initial value) were not considered.

3. Corrected Approach

(a) Simple loop with maximum n = max (100):

Test	Iterations	Purpose
1	0 (n = 0)	Skip the loop completely
2	1	Single pass
3	2	Repetition and data carried between iterations
4	m (for example 50, where 2 < m < max)	Typical case
5	max - 1 (99)	Just below the limit
6	max (100)	Limit
7	max + 1 (101)	Just above the limit: error handling

(b) Nested loops (Beizer's method):

- Start with the innermost loop; set all outer loops to their minimum values.
- Run the simple-loop tests (min, min + 1, typical, max - 1, max, and one outside value) on the innermost loop, while the outer loops are kept at minimum iteration values.
- **Move outward one loop at a time:** test the next loop with the simple-loop values while the inner loops are at typical values and the outer ones at the minimum.
- Finally, test with all loops at maximum values and at combinations of minimum and maximum.
- Concatenated loops that are independent are tested as simple loops; dependent ones are tested as nested loops.

4. Justification (Path Integrity)

- Every path of the loop in the CFG is exercised: loop skipped, entered, repeated and exited at the limit, so no path is left unverified.
- Off-by-one defects at both ends of the iteration range (0 and max + 1) are detected.
- The nested-loop method avoids an explosion of tests (it needs a number of tests that grows linearly with the nesting levels, not exponentially) while still covering boundaries at each level.
- Termination, state carried between iterations and error handling are all verified, which ensures the integrity of the paths through the loop.

Question 9

An engineer utilizes Error Guessing as the primary testing technique for a safety-critical braking system, ignoring structured black box methods. Identify the errors in relying solely on ad-hoc techniques for high-risk modules. Analyze the impact on test adequacy. Suggest a combination of BVA and State Transition Testing to fix the strategy, justifying the increased safety assurance.

Answer

1. Errors in Relying Solely on Error Guessing

- Error guessing depends on the experience and intuition of one person; it is subjective, not systematic and not repeatable.

- **There are no coverage criteria:** it is impossible to say which requirements, input ranges, states or transitions were tested.
- Rare but dangerous combinations (sensor fault during emergency braking, release of the pedal while the anti-lock system is active) are not likely to be guessed.
- No traceability to safety requirements, and no documented evidence for certification and audit (for example ISO 26262).
- Structured techniques (equivalence partitioning, boundary value analysis, decision tables, state transition) were ignored although the system has clear ranges and states.

2. Impact on Test Adequacy

Test adequacy cannot be demonstrated: the tests may leave threshold values (for example brake pressure limits, wheel slip threshold and response time) and invalid state transitions untested. The result is a high probability of residual critical defects, no measure of confidence and no objective exit criterion. For a safety-critical system, where a failure may cause injury or death, this is unacceptable. Error guessing is valuable only as a supplement to structured techniques.

3. Corrected Strategy: BVA + State Transition Testing

(a) Boundary Value Analysis (assumed parameters):

Parameter	**Requirement**	**Boundary test values**
Pedal pressure	0% to 100%	-1, 0, 1, 99, 100, 101
Wheel slip ratio for ABS activation	Activates at 20%	19, 20, 21
Brake response time	Not more than 100 ms	99, 100, 101
Vehicle speed for ABS operation	Above 5 km/h	4, 5, 6

(b) State Transition Testing: states are Idle, Braking, ABS Active and Fail-safe (fault).

Current state	**Event**	**Expected next state / action**
Idle	Pedal pressed	Braking: brake force applied
Braking	Wheel slip > 20%	ABS Active: pressure modulated
ABS Active	Slip normal	Braking
Braking / ABS Active	Pedal released	Idle: brakes released
Any state	Sensor fault detected	Fail-safe: warning lamp, safe braking mode
Idle	Wheel slip event (invalid)	No transition, no action
Fail-safe	Pedal released then pressed	Remains Fail-safe, braking available

Test sequences cover every state, every valid transition and invalid transitions (0-switch coverage), and chains of two transitions (1-switch coverage) for critical paths such as Idle, Braking, ABS Active, Fail-safe.

4. Justification of Increased Safety Assurance

- BVA ensures that all threshold values, where defects are most likely, are tested on both sides.

- State transition testing checks the complete behaviour of the controller, including invalid events and fault handling, which error guessing cannot cover systematically.
- Coverage can be measured (all states, all transitions, all boundaries) and reported, giving objective evidence for safety certification.
- The tests are repeatable and traceable to safety requirements; error guessing can still be used afterwards for extra scenarios.

Question 10

A tester designs a Control Flow Graph (CFG) for a module but fails to represent the "else" branches of multiple nested if statements, leading to 100% statement coverage but 0% branch coverage on error paths. Analyze the errors in graph construction. Identify the root causes for the missing nodes. Propose a corrected CFG and independent paths to ensure branch adequacy.

Answer

Sample module (ATM withdrawal) with nested if statements, in which the else parts are implicit (no else code is written):

```
1  read(pin, amt);
2  if (pinValid(pin))
3      if (amt <= balance)
4          if (amt <= dailyLimit)
5              dispense(amt);
6  end;
```

1. Errors in Graph Construction

- The three decision nodes (lines 2, 3 and 4) were drawn with only the "true" edge; the "false" edge of each decision, going straight to the exit node, is missing.
- The tester drew a sequence 1-2-3-4-5-6, so the graph appears to be a straight line with no alternative path.
- Edges that skip statements (invalid PIN, insufficient balance, limit exceeded) are absent, so the error paths do not exist in the model.
- Cyclomatic complexity is wrongly calculated as 1 instead of 4.
- **Consequence:** one test (valid PIN, amount within balance and limit) executes every statement, i.e. 100% statement coverage, but none of the three false outcomes (error paths) was executed, 0% coverage of the error branches (3 of 6 branch outcomes in total, i.e. 50%).

2. Root Causes of the Missing Nodes and Edges

- **Implicit else:** the false path of an "if" without else contains no statement, so the tester, who builds the graph from statements, saw nothing to draw.
- A decision node must have two outgoing edges (true and false) whatever the code; this basic CFG rule was not applied.
- Nested conditions were treated as a single sequence, instead of as separate decision nodes.
- Concentration on the "happy path" and on statement coverage as a goal.

- No cross-check with V(G) = E - N + 2 or with the number of decisions + 1, and no peer review of the graph.

3. Corrected CFG

Node	Content	Outgoing edges
N1	read(pin, amt)	N2
N2	Decision: PIN valid?	True: N3; False: N6
N3	Decision: amt <= balance?	True: N4; False: N6
N4	Decision: amt <= dailyLimit?	True: N5; False: N6
N5	dispense(amt)	N6
N6	end (exit)	-

Nodes N = 6, edges E = 8 (N1-N2, N2-N3, N2-N6, N3-N4, N3-N6, N4-N5, N4-N6, N5-N6). V(G) = E - N + 2 = 8 - 6 + 2 = 4, which equals decisions + 1 = 3 + 1.

4. Independent Paths and Test Cases for Branch Adequacy

Path	Nodes	Test data	Branch covered	Expected result
P1	N1-N2-N3-N4-N5-N6	Valid PIN, amt 2,000, balance 10,000, limit 20,000	All three true branches	Cash dispensed
P2	N1-N2-N3-N4-N6	Valid PIN, amt 25,000, balance 50,000, limit 20,000	N4 false	Daily limit exceeded
P3	N1-N2-N3-N6	Valid PIN, amt 5,000, balance 1,000	N3 false	Insufficient balance
P4	N1-N2-N6	Invalid PIN	N2 false	PIN rejected

Justification: the four independent paths exercise all six branch outcomes (100% branch coverage), including all error paths, which gives branch adequacy and, in this case, the complete set of paths. The corrected graph also shows that statement coverage with P1 alone was inadequate, because it left the error handling of the ATM untested.